

INSTALLATION GUIDE

Installing Whisper

Using faster-whisper — Windows

AUTHOR

E. Ryckewaert
Univ. Lille, CNRS, UMR 9193 — SCALab
Cognitive and Affective Sciences
F-59000 Lille, France

READ THIS FIRST

Everything runs **on your own computer**: neither the recordings nor the transcripts are sent to a server. You install once; after that, transcribing a folder of interviews takes a single command.

Contents

1	Why this method	6	Understanding the script's settings
2	Requirements	7	Optional: identifying who speaks
3	Installation in five steps	8	Data confidentiality
4	What the script produces	9	Known limitations
5	The script to copy — <i>transcrire.py</i>	10	Troubleshooting

1 Why this method

This method uses Python and pip directly, without going through Anaconda, which is heavier and prone to download problems depending on how powerful the computer is. It is simpler to set up for occasional transcription work, and it runs **without a dedicated graphics card**: processing happens on the CPU.

SETUP TIME	AFTER THAT	WHAT YOU GET
≈ 30 min the first time, including the model download	One click per folder of recordings	Three files per recording: plain text, timestamped text, Word

2 Requirements

2.1 — Python (version 3.9 to 3.13)

Download it from python.org.

THE ONE BOX YOU MUST TICK

During installation, tick “**Add python.exe to PATH**”.

2.2 — ffmpeg

Required to read audio files. In the command prompt, copy the following:

```
winget install ffmpeg
```

Then check both respond, by copying: `python --version` then `ffmpeg -version`

3

Installation in five steps

01

Create a working folder

For example named “whisper”, and put all the audio files you want to transcribe inside it.

02

Open a command prompt in that folder

Type cmd in the File Explorer address bar. If that does not work, copy this straight into the command prompt instead: pushd "path\to\folder"

03

Install the dependencies, to be copied into the command prompt:

```
python -m pip install faster-whisper python-docx tqdm
```

04

Create a text file in that same folder: transcribe.py

In that text file, paste the code given below exactly as it is.

05

Start the transcription by copying this into the command prompt:

```
python transcribe.py
```

To stop part-way through: Ctrl + C in the command prompt.

A NOTE ON THE CODE

The scripts below are kept **identical to the French edition of this guide** — variable names, file names and console messages are in French — so that both versions describe exactly the same files. Only the explanations are translated.

4

What the script produces

The script scans the working folder and transcribes **every** audio file it finds (.mp3 .wav .m4a .mp4 .flac). For each one it writes three files, named after the original:

FILE	CONTENTS
FileName_transcription.txt	Plain text, no timestamps.
FileName_transcription_horodatee.txt	Text with time markers [00:03 - 00:12], to locate a specific passage in the audio.
FileName_transcription.docx	Word version, ready to drop into a working document.

A progress bar runs during processing; two beeps signal the end of the whole folder – an idea from Dr

Louise Zanni.

5

The script to copy

transcribe.py

```
import os
import glob
from datetime import timedelta
from faster_whisper import WhisperModel
from tqdm import tqdm
import winsound
from docx import Document

MODELE = "large-v3" # plus precis que "medium", mais plus lent sur CPU
DOSSIER = "." # dossier courant : le script traite le dossier ou il est lance
EXTENSIONS = ("*.mp3", "*.wav", "*.m4a", "*.mp4", "*.flac")

def format_temps(secondes):
    td = timedelta(seconds=secondes)
    total_sec = int(td.total_seconds())
    h, reste = divmod(total_sec, 3600)
    m, s = divmod(reste, 60)
    if h > 0:
        return f"{h:02d}:{m:02d}:{s:02d}"
    return f"{m:02d}:{s:02d}"

def transcribe_fichier(model, chemin_audio):
    nom_base = os.path.splitext(os.path.basename(chemin_audio))[0]
    print(f"\n=== Transcription de : {chemin_audio} ===")

    segments, info = model.transcribe(
        chemin_audio,
        language="fr",
        vad_filter=True,
        vad_parameters=dict(min_silence_duration_ms=500),
        no_speech_threshold=0.4,
        compression_ratio_threshold=2.8,
        condition_on_previous_text=False,
        beam_size=5,
    )
    print(f"Langue detectee : {info.language} (probabilite {info.language_probability:.2f})")
    print(f"Duree totale : {info.duration:.1f}s")

    texte_simple = ""
    texte_horodate = ""

    try:
        pbar = tqdm(total=info.duration, unit="s", desc="Transcription",
                    bar_format="{l_bar}{bar}| {n:.0f}/{total:.0f}s", ncols=80)
    except Exception:
        pbar = None

    for segment in segments:
        texte_simple += segment.text
        debut = format_temps(segment.start)
        fin = format_temps(segment.end)
        texte_horodate += f"[{debut} - {fin}] {segment.text.strip()}\n"
        if pbar is not None:
            try:
                pbar.update(segment.end - pbar.n)
            except Exception:
                pbar = None
```

```
if pbar is not None:
    try:
        pbar.close()
    except Exception:
        pass

fichier_simple = f"{nom_base}_transcription.txt"
fichier_horodate = f"{nom_base}_transcription_horodate.txt"
fichier_docx = f"{nom_base}_transcription.docx"

with open(fichier_simple, "w", encoding="utf-8") as f:
    f.write(texte_simple.strip())

with open(fichier_horodate, "w", encoding="utf-8") as f:
    f.write(texte_horodate.strip())

doc = Document()
doc.add_heading(nom_base, level=1)
doc.add_paragraph(texte_simple.strip())
doc.save(fichier_docx)

print(f"-> {fichier_simple}")
print(f"-> {fichier_horodate}")
print(f"-> {fichier_docx}")

def main():
    model = WhisperModel(MODELE, device="cpu", compute_type="int8")

    fichiers_audio = []
    for ext in EXTENSIONS:
        fichiers_audio.extend(glob.glob(os.path.join(DOSSIER, ext)))

    if not fichiers_audio:
        print("Aucun fichier audio trouve dans ce dossier.")
        return

    print(f"{len(fichiers_audio)} fichier(s) audio trouve(s) :")
    for f in fichiers_audio:
        print(f" - {f}")

    for chemin_audio in fichiers_audio:
        transcrire_fichier(model, chemin_audio)

    print("\nToutes les transcriptions sont terminees.")
    winsound.Beep(1000, 200)
    winsound.Beep(1200, 200)

if __name__ == "__main__":
    main()
```

Save as transcrire.py in the working folder.

6

Understanding the settings

Nothing needs changing for the script to work. This table is here to help you understand the lines and know *what* to change if you need to.

LINE	WHAT IT DOES
<code>MODELE = "large-v3"</code>	The model used. The larger it is (<i>tiny</i> → <i>small</i> → <i>medium</i> → <i>large-v3</i>), the more accurate and the slower on CPU. Switch to "medium" for faster processing, at a slight cost in accuracy.
<code>DOSSIER = "."</code>	The dot means “current folder”: the script processes the folder it is launched from.
<code>glob.glob(...)</code>	Builds the list of audio files in the folder automatically: no file name has to be typed in by hand.
<code>vad_parameters</code> <code>no_speech_threshold</code> <code>compression_ratio_threshold</code>	Reduce words dropped or wrongly “corrected” — for instance colloquial French tidied into its written form. They curb the model's tendency to smooth informal speech towards written language.
<code>condition_on_previous_text=False</code>	Stops the model “guessing” what comes next from the text already transcribed, which cuts down automatic rephrasing.
<code>try / except around tqdm</code>	Guards against a known progress-bar bug in some Windows terminals, which can otherwise kill the script just before the files are written.
<code>doc.add_paragraph(...)</code> <code>doc.save(...)</code>	Generate the Word version of the plain text.
<code>winsound.Beep(1000, 200)</code>	Sounds the finishing signal once every file has been processed.

7 Optional: identifying who speaks

Diarization

THIS IS AN OPTION, NOT A REQUIRED STEP

The script in section 5 is enough to obtain a complete transcript. This section is only useful if you need to know **who says what** — for example telling apart two or three people speaking in the same interview.

Setting this up takes longer than the base script, because it relies on third-party models (pyannote.audio) whose access is controlled individually. In practice, **each person** must create their own free **Hugging Face** account, accept the terms of three models, and generate their own access token. None of it can be passed from one person to another.

This setup happens **once per person**. After that, day-to-day use is as simple as the base script.

7.1 — One-time setup

01 Create a free account

On huggingface.co.

02 Accept the terms of three models

The “Agree and access repository” button on each of these pages:

huggingface.co/pyannote/speaker-diarization-3.1

huggingface.co/pyannote/segmentation-3.0

huggingface.co/pyannote/speaker-diarization-community-1

03 Generate an access token

At huggingface.co/settings/tokens, type **Read**.

04 Install the extra library

```
python -m pip install pyannote.audio
```

05 Store the token without writing it into a file

```
setx HF_TOKEN "paste_the_token_here_by_right_click"
```

Then **close the window and open a new one**: setx only takes effect in windows opened after the command. Check with `echo %HF_TOKEN%`.

DO NOT ROUTE THE TOKEN THROUGH WORD

Copy-pasting through a word processor such as Word can silently corrupt characters in the token: visually identical but technically different letters get substituted. Go through Notepad, which changes nothing, and paste the token from there.

7.2 — The script with speakers

Save it as `transcrire_locuteurs.py`. It contains the whole base script — same settings, same progress bar, same three outputs — plus a fourth output with the speakers. **Nothing to edit:** the token is read automatically from the environment variable set above.

```
import os
import glob
from datetime import timedelta
import numpy as np
import torch
import av
from faster_whisper import WhisperModel
from pyannote.audio import Pipeline
from tqdm import tqdm
from docx import Document
import winsound

MODELE_WHISPER = "large-v3"
DOSSIER = "." # dossier courant
EXTENSIONS = ("*.mp3", "*.wav", "*.m4a", "*.mp4", "*.flac")

# Le token est lu depuis la variable d'environnement HF_TOKEN
# (definie une fois avec la commande setx), jamais stocke dans ce fichier.
HF_TOKEN = os.environ.get("HF_TOKEN")

# Nombre de personnes qui parlent dans l'audio.
# Mets 2 si c'est toujours un entretien a deux (aidant + personne malade),
# ou laisse None pour que le programme le devine automatiquement (moins fiable).
NOMBRE_LOCUTEURS = 2
FREQUENCE_CIBLE = 16000 # frequence attendue par le modele de diarisation

def format_temps(secondes):
    td = timedelta(seconds=secondes)
    total_sec = int(td.total_seconds())
    h, reste = divmod(total_sec, 3600)
    m, s = divmod(reste, 60)
    if h > 0:
        return f"{h:02d}:{m:02d}:{s:02d}"
    return f"{m:02d}:{s:02d}"

def charger_audio_pour_diarisation(chemin_audio, frequence_cible=FREQUENCE_CIBLE):
    """
    Decode le fichier audio nous-memes avec la bibliotheque 'av'
    (deja installee via faster-whisper), pour eviter de dependre
    de torchcodec / FFmpeg partage qui pose probleme sous Windows.
    """
    conteneur = av.open(chemin_audio)
    flux_audio = conteneur.streams.audio[0]
    reechantillonneur = av.audio.resampler.AudioResampler(
        format="s16", layout="mono", rate=frequence_cible
    )
```

```

morceaux = []
for frame in conteneur.decode(flux_audio):
    frame.pts = None
    for frame_reechantillonnee in reechantillonneur.resample(frame):
        morceaux.append(frame_reechantillonnee.to_ndarray())
conteneur.close()

audio_np = np.concatenate(morceaux, axis=1).astype(np.float32) / 32768.0
waveform = torch.from_numpy(audio_np)
return waveform, frequence_cible

def extraire_annotation(resultat_diarisation):
    """
    Les versions recentes de pyannote.audio renvoient un objet qui encapsule
    le resultat plutot que de le renvoyer directement. Cette fonction
    retrouve l'annotation utilisable quel que soit le format renvoye.
    """
    if hasattr(resultat_diarisation, "itertracks"):
        return resultat_diarisation
    for attribut in ("speaker_diarization", "annotation", "diarization"):
        if hasattr(resultat_diarisation, attribut):
            return getattr(resultat_diarisation, attribut)
    print("ATTENTION : format inconnu. Attributs disponibles :", dir(resultat_diarisation))
    raise AttributeError("Impossible de trouver l'annotation dans le resultat de la diarisation.")

def assigner_locuteur(debut, fin, annotation):
    meilleur_locuteur = None
    meilleur_overlap = 0
    for turn, _, locuteur in annotation.itertracks(yield_label=True):
        overlap = min(fin, turn.end) - max(debut, turn.start)
        if overlap > meilleur_overlap:
            meilleur_overlap = overlap
            meilleur_locuteur = locuteur
    return meilleur_locuteur or "Locuteur inconnu"

def transcrire_fichier(model_whisper, pipeline_diarisation, chemin_audio):
    nom_base = os.path.splitext(os.path.basename(chemin_audio))[0]
    print(f"\n=== Traitement de : {chemin_audio} ===")

    print("Etape 1/2 : transcription du texte...")
    segments, info = model_whisper.transcribe(
        chemin_audio,
        language="fr",
        vad_filter=True,
        vad_parameters=dict(min_silence_duration_ms=500),
        no_speech_threshold=0.4,
        compression_ratio_threshold=2.8,
        condition_on_previous_text=False,
        beam_size=5,
    )
    print(f"Langue detectee : {info.language} (probabilite {info.language_probability:.2f})")
    print(f"Duree totale : {info.duration:.1f}s")

    texte_simple = ""
    texte_horodate = ""
    segments_liste = []

    try:
        pbar = tqdm(total=info.duration, unit="s", desc="Transcription",
                    bar_format="{l_bar}{bar}| {n:.0f}/{total:.0f}s", ncols=80)
    except Exception:

```

```

pbar = None

for segment in segments:
    segments_liste.append(segment)
    texte_simple += segment.text
    debut = format_temps(segment.start)
    fin = format_temps(segment.end)
    texte_horodate += f"[{debut} - {fin}] {segment.text.strip()}\n"
    if pbar is not None:
        try:
            pbar.update(segment.end - pbar.n)
        except Exception:
            pbar = None

if pbar is not None:
    try:
        pbar.close()
    except Exception:
        pass

print("Etape 2/2 : identification des locuteurs (peut prendre plusieurs minutes)...")
waveform, frequence = charger_audio_pour_diarisation(chemin_audio)
resultat_diarisation = pipeline_diarisation(
    {"waveform": waveform, "sample_rate": frequence},
    num_speakers=NOMBRE_LOCUTEURS,
)
annotation = extraire_annotation(resultat_diarisation)

texte_locuteurs = ""
dernier_locuteur = None
for segment in segments_liste:
    locuteur = assigner_locuteur(segment.start, segment.end, annotation)
    if locuteur != dernier_locuteur:
        texte_locuteurs += f"\n\n[{format_temps(segment.start)}] {locuteur} :\n"
        dernier_locuteur = locuteur
    texte_locuteurs += segment.text.strip() + " "

fichier_simple = f"{nom_base}_transcription.txt"
fichier_horodate = f"{nom_base}_transcription_horodate.txt"
fichier_docx = f"{nom_base}_transcription.docx"
fichier_locuteurs = f"{nom_base}_transcription_locuteurs.txt"

with open(fichier_simple, "w", encoding="utf-8") as f:
    f.write(texte_simple.strip())

with open(fichier_horodate, "w", encoding="utf-8") as f:
    f.write(texte_horodate.strip())

with open(fichier_locuteurs, "w", encoding="utf-8") as f:
    f.write(texte_locuteurs.strip())

doc = Document()
doc.add_heading(nom_base, level=1)

# Le Word integre directement les tours de parole (locuteur + horodatage),
# plutot que le texte brut sans distinction des interlocuteurs.
blocs = [b.strip() for b in texte_locuteurs.strip().split("\n\n") if b.strip()]
for bloc in blocs:
    lignes = bloc.split("\n", 1)
    entete = lignes[0].strip()
    texte_bloc = lignes[1].strip() if len(lignes) > 1 else ""

    p_entete = doc.add_paragraph()
    run_entete = p_entete.add_run(entete)
    run_entete.bold = True

```

```
        if texte_bloc:
            doc.add_paragraph(texte_bloc)

    doc.save(fichier_docx)

    print(f"-> {fichier_simple}")
    print(f"-> {fichier_horodate}")
    print(f"-> {fichier_docx}")
    print(f"-> {fichier_locuteurs}")

def main():
    if not HF_TOKEN:
        print("ERREUR : la variable d'environnement HF_TOKEN n'est pas definie.")
        print('Rappel : setx HF_TOKEN "ton_token" dans une invite de commandes, puis ouvrir une NOUVELLE fenetre.')
        return

    print("Chargement des modeles (transcription + diarisation)...")
    model_whisper = WhisperModel(MODELE_WHISPER, device="cpu", compute_type="int8")
    pipeline_diarisation = Pipeline.from_pretrained(
        "pyannote/speaker-diarization-3.1",
        token=HF_TOKEN,
    )

    fichiers_audio = []
    for ext in EXTENSIONS:
        fichiers_audio.extend(glob.glob(os.path.join(DOSSIER, ext)))

    if not fichiers_audio:
        print("Aucun fichier audio trouve dans ce dossier.")
        return

    print(f"{len(fichiers_audio)} fichier(s) audio trouve(s) :")
    for f in fichiers_audio:
        print(f" - {f}")

    for chemin_audio in fichiers_audio:
        transcrire_fichier(model_whisper, pipeline_diarisation, chemin_audio)

    print("\nTous les fichiers ont ete traites.")
    winsound.Beep(1000, 200)
    winsound.Beep(1200, 200)

if __name__ == "__main__":
    main()
```

Run with: `python transcrire_locuteurs.py`

7.3 — What this script adds

LINE	WHY IT IS THERE
HF_TOKEN = os.environ.get("HF_TOKEN")	Reads the token from the Windows environment variable rather than from the file: no risk of corruption, and no accidental sharing (public GitHub repository, script sent by email).
charger_audio_pour_diarisation(...)	Decodes the audio with the av library instead of letting pyannote do it. Works around a common Windows bug (“Could not load libtorchcodec”).
extraire_annotation(...)	pyannote.audio has changed the structure of its result across versions. This function finds the usable information whatever the format, and prints a clear message if a new one appears.
NOMBRE_LOCUTEURS = 2	Set it to the real number of people speaking. Use None if that number varies between files — automatic detection, but less reliable.
blocs = [...] run_entete.bold = True	Write the Word file turn by turn , speaker and timestamp in bold, then the text, instead of one undifferentiated block of prose. This is the version to proofread in Word.

7.4 — The fourth output

On top of the three usual files, a `FileName_transcription_locuteurs.txt` is written, organised by speaker turn:

```
[00:00] SPEAKER_00 :
Bonjour, merci d'être venus...

[00:14] SPEAKER_01 :
Je vous en prie...
```

TWO THINGS TO KNOW

The labels (SPEAKER_00, SPEAKER_01...) are generic: the program tells voices apart but has no idea who is who. Work out yourself, at the top of each file, which label belongs to which person — **the order can change from one file to the next**.

Processing is markedly slower than a plain transcription: diarization adds a full analysis of the audio. Without a dedicated graphics card, expect considerably longer than the real length of the recording.

8 Data confidentiality

Both scripts process the audio **entirely locally**, on the computer that runs them. Neither the audio files nor the transcribed text are sent to an external server during processing — unlike online transcription services (Otter.ai, the OpenAI API, and so on).

One exception: on the very first run, the models used (Whisper, and pyannote for the diarization option) are downloaded from Hugging Face. That download only involves the model's generic files — never your data.

CHECK BEFORE USING THIS ON RESEARCH DATA

That participant consent covers the use of an automatic transcription tool

That the audio files and transcripts stay in the location approved by the lab.

That any identifying information spoken aloud — names, places — is anonymised manually after transcription: **the tool does not do it**

9 Known limitations

- 01** The first run downloads the chosen model, up to roughly **3 GB** for *large-v3*, more with diarization. A stable connection is needed. Later runs reuse the models already downloaded.
- 02** Diarization requires an individual Hugging Face account; it cannot be made available without that step, because of the models' licensing.
- 03** CPU processing is inherently slow: plan to launch a large folder at the end of the day.

10 Troubleshooting

MESSAGE OR SYMPTOM	WHAT TO DO
'python' is not recognized	The “Add python.exe to PATH” box was not ticked (§2.1). Reinstall Python with it ticked.
Aucun fichier audio trouve dans ce dossier	The command prompt is not open in the right folder, or the extensions are not in the list. Check with <code>dir</code> .
Could not load libtorchcodec	pyannote's audio-loading bug on Windows. The script in §7.2 already works around it: make sure you are using that version.
HF_TOKEN n'est pas definie	The <code>setx</code> command ran, but the window was not reopened. Close it, open a new one, check with <code>echo %HF_TOKEN%</code> .
Hugging Face authentication fails despite a valid token	The token was most likely damaged by a trip through a word processor such as Word. Regenerate it and paste it from Notepad straight into <code>setx</code> .
Access denied to a pyannote model	The terms of all three pages have not been accepted (§7.1, step 02). They are agreed one at a time, with the account that generated the token.

An error not listed here, a setting that has changed? Report it for the next version of this guide.

END OF GUIDE